In [1]:
```
import os
import re
import pickle
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
import glob
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
pd.set_option('display.max_colwidth', None)
```

Dataset Choice

In [2]:
```
json_dir = '../results/2024/'
save_db = 'windows_2024.pkl'
```

Retrieve Results

In [3]:
```
flattened_data = []
json_files = glob.glob(os.path.join(json_dir, '*.json'))
for json_file in json_files:
    with open(json_file, 'r') as f:
        try:
            data = json.load(f)
            for author_data in data:
                author = author_data['author']
                for window in author_data['windows']:
                    record = {
                        'author': author,
                        'start_index': window['start_index'],
                        'start_time': window['start_time'],
                        'end_index': window['end_index'],
                        'end_time': window['end_time'],
                        'timezones': window['timezones']
                    }
                    flattened_data.append(record)
        except Exception as e:
            print(f"Error processing file {json_file}: {e}")

df = pd.DataFrame(flattened_data)
```

```
Error processing file ../results/2024/af1aa96f31a617f882a08a20d921fd165a7c0
c3957cdbc566ea1541cefde0545.json: Expecting ',' delimiter: line 1 column 21
47479553 (char 2147479552)
```

In [4]:
```
starts = pd.to_datetime(df['start_time'])
ends = pd.to_datetime(df['end_time'])
df['duration'] = (ends - starts).dt.total_seconds() / 3600
```

Retrieving time: **7min**

Saving and Loading Results

In [5]:
```
with open(save_db, 'wb') as f:
    pickle.dump(df, f)
```

Saving time: **3min**

In [6]:
```
with open(save_db, 'rb') as f:
    df = pickle.load(f)
```

Loading time: **2min**

In [7]:
```
df.head()
```

Out[7]:

	author	start_index	s
0	ce78d68d19d779994948d1eee959ab5d14c673b19ba33874be194c4996151139	325	25T
1	ce78d68d19d779994948d1eee959ab5d14c673b19ba33874be194c4996151139	485	19T
2	ce78d68d19d779994948d1eee959ab5d14c673b19ba33874be194c4996151139	507	19T
3	ce78d68d19d779994948d1eee959ab5d14c673b19ba33874be194c4996151139	527	27T
4	ce78d68d19d779994948d1eee959ab5d14c673b19ba33874be194c4996151139	533	28T

Analysis

Basic Stats

In [8]:
```
author_stats = pd.DataFrame({
    'total_windows': df.groupby('author').size(),
    'unique_patterns': df.groupby('author')['timezones'].apply(lambda x: len
    'avg_changes_per_window': df.groupby('author')['timezones'].apply(lambda
    'avg_window_durations': df.groupby('author')['duration'].mean()
})
```

2min

In [9]:
```
author_stats.head(2)
```

Out[9]:

	total_windows
author	
000004ce1e8b2d27a17eff682ee038222519d0139e2e870128c713ef46ab1f20	1
000014b10fb13559dfd44bc447b49334ffbbf793f8859e478bd369d384cdc013	8

In [10]:
```
max_stats = {
    'total_windows': author_stats['total_windows'].idxmax(),
    'unique_patterns': author_stats['unique_patterns'].idxmax(),
    'avg_changes_per_window': author_stats['avg_changes_per_window'].idxmax(
    'avg_window_durations': author_stats['avg_window_durations'].idxmin()
}
```

```
max_authors = pd.DataFrame({
    'statistic': max_stats.keys(),
    'author': max_stats.values(),
    'value': [author_stats.loc[max_stats[stat], stat] for stat in max_stats
})
# what date for unique pattern ? Old ?
max_authors
```

Out[10]:

	statistic	au
0	total_windows	d2833f622cce50d1194cabd68ab1f50be3f55e0d07acf37e525daae9ee13
1	unique_patterns	d2833f622cce50d1194cabd68ab1f50be3f55e0d07acf37e525daae9ee13
2	avg_changes_per_window	d7dfa50f20e7ffe1e3c76ef59c307de208bd5359ad161e691d6a2a9b724e
3	avg_window_durations	0001a58dc89c12e4ba2fdf35a0e0b68f875aca61f71178adce47c2020505

- a26a80aeb4898a459a3a870a9ebbc990fab48497417ff160ee90e5528b202744 -> Jeff Epler jepler@gmail.com
- a2043a2e733a2cf67eccfa901584bd9ad4b60a79816a76a87604125bffd8f6d1 -> shaun shaundindial030@gmail.com
- a1f7e48ff270892792bbbea3eba2553b98cdba80d1c89252c144e8360a352ebb -> rOpenSci Bot myrmecocystus+ropenscibot@gmail.com

Effective window

In [13]:

```
avg_duration = author_stats['avg_window_durations']
smallest_durations = avg_duration.sort_values().head(10)
print("Top 10 authors with shortest average window duration")
print(smallest_durations)
# 0.0 is probably only bots
filtered_durations = avg_duration[(avg_duration >= 1) & (avg_duration < 2)]
print("Authors with average window duration between 1–2 hours:")
print(filtered_durations.head(10))
```

```
Top 10 authors with shortest average window duration
author
7182a4dd77d5826c85df9d461b6cb2d350b3ae7aadae035b42a2b935d3c6c276    0.0
9aab543bd71e2dce043d4f87e83f718ad656f5131977d943d064fe3a9f1b2067    0.0
63d3fdfedad12b848bb9dfaa7b92863e74b5b9948c54bc29d7112a86bb4537ed    0.0
16f596c65316e2a00d5ef3088ca824ce8740ad642784282605a462d283427f14    0.0
16f54d71b09477c8bbec329382b2b34bae323833814371c73879ffecb6bf6a23    0.0
76953e511a84370e4f46c40167c4219eaf885b33501d7b7a4332df464ea035ae    0.0
16f47a91219c5630d8c42533832fff8f3b990915aad0e1a2c61d8d42042a58fc    0.0
3071f5ae32109640256d45825494354259d62c7ebebc1bd6c44655fa81bedbb6    0.0
a7bc5822ceaace6c2842f131829f209cc7fdd7265e316aaf8c9c9e8a2a253032    0.0
f4a5244fc16de83943836b2437670cb452aefd37604d10b795cd006256118742    0.0
Name: avg_window_durations, dtype: float64
Authors with average window duration between 1-2 hours:
author
0002b5c7d16d92cb43bc61b16aed68e10d7f633bddb29543d44e7acd96af2ca4    1.39138
9
00032c2b7ee8862c4b40cf64029772b08d0b783e6250fa0e73038d367f11a6a5    1.39638
9
00052b1f9b9a84ae400e8eedcfd25ee140597dc1ff7e92531f6ea0b20d999ab0    1.20319
4
0007520bc9a799b326fe5e2b3550a4ebae76adb39e172caa47925bf3bdace7d9    1.30027
8
000917c70fe1221257bc16fcb95da0105aae632d73149f5ac70032e911328534    1.52083
3
0009746d80f93fe5260e8b6f1820b2d0af1223e7487631f1eae546dbb33fc379    1.63083
3
0009a975e170468278e15764e9ef856ac6afedec39c72f9e0b42c59fca5a067b    1.52263
9
000a96f22bd3ba154a6d22d0eed5dbd66ce9d98af1d2eca589c8a0cafb033943    1.94826
4
000b20181110cd978c3fb93859f44138a29f98ea13569490d2d4a175af1f53f6    1.25944
4
000bb292cb6d721bf5c664fae5008d9df4f1ae17f338272c587e2ee862b864dd    1.09888
9
Name: avg_window_durations, dtype: float64
```

- a09227853d1dc6b05f5766c8ed052c0a0ef028dcba05339a4151e405affc446b 1.785556h -> jvivas jovies288@gmail.com
  - [{'start_index': 1, 'start_time': 1384313409, 'end_index': 9, 'end_time': 1384319837, 'timezones': [-360, -480, -360]}]

Timezone Pattern

In [14]:
```
timezone_patterns = df.groupby('author')['timezones'].apply(
    lambda x: [tuple(w) for w in x]
)
common_patterns = pd.Series([p for patterns in timezone_patterns for p in pa
```

2min

In [15]:
```
long_patterns = [(pattern, common_patterns[pattern]) for pattern in common_p
sorted_long_patterns = sorted(long_patterns, key=lambda x: x[1], reverse=Tru
print("Top 10 most frequent patterns with more than 2 unique timezones:")
for pattern, count in sorted_long_patterns[:10]:
    print(f"Pattern: {pattern}, Count: {count}")
```

```
Top 10 most frequent patterns with more than 2 unique timezones:
Pattern: (-420, 180, 120), Count: 1308
Pattern: (-420, 120, 180), Count: 1219
Pattern: (60, 0, 120), Count: 1156
Pattern: (120, 0, 60), Count: 1068
Pattern: (-240, 0, -420), Count: 1067
Pattern: (120, 60, 0), Count: 1002
Pattern: (120, 0, -420), Count: 978
Pattern: (-420, 0, 120), Count: 952
Pattern: (-420, -240, 0), Count: 920
Pattern: (60, 120, 0), Count: 907
```

4minmin

Graphic

In [16]:
```
""" plt.figure(figsize=(10,6))
sns.histplot(data=[d for durations in window_durations for d in durations if
plt.title('Window Durations Distribution')
plt.xlabel('Hours') """

plt.figure(figsize=(12,6))
common_patterns.head(10).plot(kind='bar')
plt.title('Most Common Timezone Patterns')
plt.xticks(rotation=45)
plt.tight_layout()
```

